参考编译器介绍

本实验将基于C语言编译器的结构，介绍从词法分析到中间代码优化的过程。虽然编译器通常包含多个阶段，但本实验将重点关注词法分析、语法分析、中间代码生成及优化四个阶段.

1.总体结构

编译器通常由多个阶段组成，每个阶段负责将源代码转换成目标机器代码。编译器的主要阶段包括:

词法分析：将源代码中的字符流转换成一系列的词法单元（Token）。

语法分析：根据语言的语法规则，将Token序列转换成抽象语法树（AST）。

中间代码生成：将抽象语法树转换成平台无关的中间代码。

优化 ： 对生成的中间代码进行优化，提高代码的执行效率和性能。

汇编 ： 将优化后的中间代码转换成机器代码。

链接 ： 将多个目标文件合并成一个可执行文件。

本实验将专注于词法分析到优化这几个阶段，展示如何通过每个步骤将源代码逐步转化为中间代码。

2. 词法分析

词法分析是编译过程的第一步，任务是将源代码中的字符流转换为一个个有意义的词法单元（Token）。每个Token代表源代码中的一个最小语法单元，如关键字、标识符、运算符、常量等。

词法分析器通过正则表达式和有限自动机来识别Token。

生成的Token序列会被传递给语法分析器进行后续处理。

3. 语法分析

语法分析的任务是根据编程语言的语法规则，将词法分析器产生的Token序列转换为抽象语法树（AST）。抽象语法树是一种树状结构，其中每个节点代表一个语法元素（如表达式、语句、声明等）。

语法分析器会根据文法规则检查Token序列是否符合语言的语法规范。如果发现错误，则会报错并退出。

语法分析的输出是抽象语法树（AST），该树是中间代码生成的基础。

4. 中间代码生成

中间代码生成阶段将抽象语法树（AST）转换为一种中间表示（Intermediate Representation，IR）。中间代码通常是与特定硬件平台无关的，可以跨平台移植，并且更易于优化。

三地址码（Three-Address Code，TAC）**是常见的中间代码形式。每条指令通常包含操作符、两个操作数以及结果变量。

中间代码生成器负责将AST中的表达式和语句转换为中间代码，生成过程可以支持不同的中间代码格式。

5. 优化

由用pcode来生成代码本次实验不做！

6. 接口设计

编译器的各个模块通过清晰的接口进行数据交换：

词法分析器(Lexer) 将源代码转换为 Token，传递给 语法分析器(Parser)。

语法分析器(Parser) 将Token流转换为 抽象语法树(AST)，并传递给 中间代码生成器。

中间代码生成器 生成中间代码，并将其传递给 优化器(Optimizer) 进行优化。

7. 文件组织

编译器的文件结构一般包括以下几个部分：

源代码文件：编写的C语言源代码文件。

头文件：包含函数声明、宏定义、数据结构等内容的文件。

词法分析器 ： 将源代码中的字符转换为Token的模块。

语法分析器 ： 根据语言的文法规则构建抽象语法树(AST)的模块。

中间代码生成器 ： 将AST转换为中间代码的模块。

编译器总体结构

本次实验使用C++编写，分析输入文件（testfile.txt）的内容，这侧计划实现的编译器遵循传统的编译器设计架构，采用模块化的设计方法。编译器将按以下顺序工作：

词法分析阶段：将源代码转化为一个个的词法单元（Token），为后续的语法分析做准备。

语法分析阶段：根据语言的文法规则，解析 Token 序列，生成抽象语法树（AST）。

语义分析阶段：检查语法树中是否存在语义错误，构建符号表并进行类型检查等。

代码生成阶段：根据抽象语法树，生成目标代码（如 Pcode）。

接口设计

Lexical_Analysis：定义词法分析器接口，包括初始化、词法分析函数等。

Syntax_Analysis：定义语法分析器接口，负责解析 Token 并生成 AST，并且定义语义分析器接口，用于符号表管理和错误检查。

Interpreter class：定义代码生成器接口，将语法树转换为目标代码。

接口之间通过标准的数据结构进行交互。

文件组织

Lexical_Analysis：包含词法分析的实现，包括初始化、Token 生成、注释处理等功能。

Syntax_Analysis：包含语法分析的实现，负责解析输入的 Token 序列并生成语法树，并实现符号表管理、语义错误检查等功能。

Interpreter class：负责将语法树转换为目标代码（如中间代码）。

main.c：作为主程序文件，负责编译流程的控制和各模块的协调。

词法分析设计

设计总结

该词法分析器为高效处理输入源代码，首先将输入的内容逐行区分存储，然后开始进行词法分析。为了实现稳定的词法分析并为未来的附加实验做好准备，代码分割成尽可能多的函数，目标是实现面向对象的设计。这使得在额外实验中能够灵活地添加类型和改善功能。

主要组成部分

1. 初始化（init() 函数）
    - 打开文件，并逐行读取内容存储到 input_text 中。
    - 计算包含换行符的输入文本总长度。
2. 词法分析主例程（Lexical_Analysis() 函数）
    - 对输入文本的每个字符执行以下操作：
        - 行号检查：检测换行符以更新全局行号 (glb_linenum)。

        - 空白和注释处理:通过skipBlank()和与注释相关的函数(skip_line_Comment() 和 skip_block_Comment()) 跳过不需要分析的字符。
        - 执行词法分析：调用 Lexical_Analysis_run() 函数，根据当前位置的字符识别令牌。
3. 执行词法分析（Lexical_Analysis_run() 函数）
    - 根据当前字符的类型进行不同处理:
        - 标识符和数字：以字母或数字开头的字符串被读取并添加到令牌列表中。
        - 符号和关键字：识别每个符号的正确令牌并添加到列表，关键字则使用 push_TypeToken() 函数特别处理。
        - 字符串：识别用双引号包围的字符串并处理为 STRCON。
        - 注释处理：跳过行注释（//）和块注释（/* */）。
        - 错误处理：记录无法识别的字符。
4. TOKEN及错误存储
    - Token列表：使用 Token_word 结构体存储每个识别令牌的信息，包括类型、实际字符串和行号。
    - 错误列表：使用 error_word 结构体存储无法识别的字符的错误信息。
5. 结果输出
    - 结果通过 output_Lexical() 函数记录到输出文件。如果发生错误，则输出错误信息；否则输出识别的令牌列表。

编码完成之后的修改

1. 输入处理错误
    - 问题：在处理多行输入时，行号更新缺失，导致错误。
    - 修正：修改函数以防止意外的换行和 glb_linenum 增加，确保适当更新 glb_linenum。
2. 注释处理的低效性
    - 问题：处理行注释和块注释的方式效率低下，且在注释处理过程中出现错误。

- 修正：优化注释处理函数，以跳过包含注释的行，从而修复错误并提高效率

语法分析设计

设计总结

首先，创建了两个可以调整Token位置的函数，之后再创建了这个编译器实验支持的所有文法的函数，然后从CompUnit开始实现了代码。实现该语法分析的函数之前分析该文法的结构，若该文法之中有多个参数的话用前面设计的调整Token位置的函数来存储该Token参数，判断后面调用的函数是什么。

按照问题要求用递归结构来设计编译器，且在每层语法分析函数中需要输出该语法成本的名称，弄好了每个语法成本的函数的末尾输出名称的逻辑。

最后要设计的是语法分析里面的错误处理，这些错误都是后面没有一个Token（例如：；，），]），所以用直观的方法来处理了。用nextToken()函数来判断后面是否有该Token。

这些设计可以满足该实验的要求，一旦开始CompUnit函数的话递归的处理testfile的内容和输出正确的结果和错误内容。

主要组成部分

1.调整Token位置的函数（preToken(),nextToken()函数）

- 执行语法分析时用这两个函数来调整Token的现在位置

2.语法分析主程序（ Syntax_Analysis()函数）

- 把Token_list的位置初始化到0，之后开始语法分析

3.个个文法的函数（parseCompUnit(),parseDecl(),parseVarDef()等）

- 函数命名为parse加上该语法的名称

4.输出结果（printSyntax()函数）

- 输出改语法的名称

编码完成之后的修改

1. 处理Exp设计错误

问题：在处理计算类的Exp时设计错误，就是设计逻辑有错

修正：由于设计的编译器是递归架构，所以在函数处理的时候先需要调用之前的函数，并加上优化。

例如：

```
//AddExp → MulExp | AddExp ('+' | '-') MulExp
void parseAddExp() {
    parseMulExp();

    printSyntax("AddExp");

    if(curToken.word == "+" || curToken.word == "-"){
        printcurToken();  // '+' | '-'
        nextToken();

        parseAddExp();
    }
}
```

2.处理语法错误时的错误

问题：我设计的逻辑在某个情况下出错。

修正：把错误处理的逻辑加强了逻辑，提高判断错误的准确性

语义分析设计

设计总结

识别符号是感觉不太难，符号产生的情况就是普通变量声明、常量变量声明和函数声明只有这三种情况，因此只需要创建符号表，每当有符号出现时，就将其插入符号表中。

在设计符号处理后，我发现这次实验的难点是错误处理。需要处理的错误种类很多，特别是与函数参数相关的错误处理，从设计开始就不容易。但是我尝试将符号接收的部分都交给符号表来处理。

其他的错误是海事并不难的，其他错误设计是如下

错误B: 检查变量是否在当前作用域（nowScope）内已经声明。如果同名的变量已经存在，则调用错误 B，防止重复声明。

错误C: 若符号表中该变量在任何作用域中都没有定义，则调用错误。

错误F: 若不是 void 类型的函数但没有 return，则调用错误。

错误G: 若需要返回值的函数没有 return，则调用错误。

错误H: 若Lval 是常量，则调用错误。

错误L: 若 printf 中 %d 和 %c 的数量与后面的参数数量不匹配，则调用错误。

错误M: 若在 for 循环外出现 break 或 continue，则调用错误。

主要组成部分

1.创建 SymbolTable 包含(string name,int type, int kind, int isconst, int socpe)

执行以下操作:

声明变量和函数时添加该识别符。

这时保证错误B的处理，由添加识别符前一定确认symboltable里面有没有该识别符

2.关于错误管理

在错误类型 B 中，我们解决了重定义错误。重定义指的是在同一块或上层块中，变量已经被声明过了，但又重新进行了定义。为了解决这个问题，我们需要检查每个变量是否在相同的块或上层块中使用。为了实现这一点，我们引入了nowScope和glb_curSocpe的概念。

```
bool parseErrorB(string name, int type, int kind, int isconst, int nowScope){
    for (int i = ST_top; i >= 0; i--) {
        if (symbolTable[i].scope == nowScope && symbolTable[i].name == name) {
            push_err_list('b');
            return false;
        }
        if(isequalScope){
            if (symbolTable[i].scope == curScope) {
                if (symbolTable[i].name == name) {
                    push_err_list('b');
                    return false;
                }
            }
        }
    }

   add_s_Table(name, type, kind, isconst);

    return true;
}
```

在错误类型 C 中，我们检查未声明的变量。为了做到这一点，我们需要检查符号表中的所有变量，以确定是否使用了未声明的变量。

```
void parseErrorC(string name) {
    bool isFound = false;
    stack<int> tempScopeStack = scopeStack;

    while (!tempScopeStack.empty()) {
        int currentScope = tempScopeStack.top(); // 현재 스코프
        printscope.push(currentScope);
        tempScopeStack.pop(); // 스택을 망가뜨리지 않고 상위 스코프로 이동

        SymbolTable* symbol = findSymbol(name, currentScope);
        if (symbol != nullptr) {
            isFound = true;
            break;
        }
    }

    if (!isFound) {
        push_err_list('c');
    }
}
```

在错误类型 E 中，我们解决要决解的是函数参数错误。如果参数设置不正确，函数会停止执行并返回错误 E。首先，我们将参数为 0 的情况作为特殊情况处理，其余情况根据参数是否为数组、是否以单一变量的形式调用数组、以及是否为普通变量、数字或字符进行分别解析。

```
void parseErrorE(int param_num) {
    funcparam& curFunc = funcparse.back();

    if (curFunc.paramcnt == 0) { // 매개변수 없을 경우 체크 X
        return;
    }

    int nextTokencnt = 0;
    bool first = true;

    while (curToken.word != ")" && curToken.word != "," && curToken.word != ";") {
        if (first) {
            first = false;
        } else {
            nextToken();
            nextTokencnt++;
        }

        // 변수일 때
        if (curToken.Token == "IDENFR") {
            findAndSetSymbol(curToken.word);
            if (curFunc.paramkind[param_num - 1] == 3) { // 기대값이 배열일 때
                nextToken(); // 배열을 단일로 받는 경우 처리
                if (curToken.word == "[") {
                    nextToken();
                    if (isdigit(curToken.word[0])) {
                        preToken();
                        preToken();
                        push_err_list('e', last_func_line);
                        break;
                    }
                    preToken();
                }
                preToken();

                if (SymbolTemp.type != curFunc.paramtype[param_num - 1] ||
                    SymbolTemp.kind != curFunc.paramkind[param_num - 1]) {
                    push_err_list('e', last_func_line);
                    break;
                }
            } else if (curFunc.paramkind[param_num - 1] == 0) { // 일반 변수
```

```
                if (curToken.word == "[") {
                    nextToken();
                    if (isdigit(curToken.word[0])) {
                        isfind = true;
                    }
                    preToken();
                }
                preToken();

                if (!isfind && SymbolTemp.kind == 3) {
                    push_err_list('e', last_func_line);
                    break;
                }
            }
        } else if (isdigit(curToken.word[0])) { // 숫자이고 배열이 아니면 인트
            printfSymbol[print_top].name = curToken.word; // print only
            printfSymbol[print_top].kind = 0;
            printfSymbol[print_top].type = 0;
            print_top++;

            if (curFunc.paramkind[param_num - 1] == 3) { // 배열 기대 시 에러 추가
                push_err_list('e', last_func_line);
                break;
            }
        } else if (isalpha(curToken.word[0]) || curToken.word[0] == '\'') {
            printfSymbol[print_top].name = curToken.word; // print only
            printfSymbol[print_top].kind = 0;
            printfSymbol[print_top].type = 1;
            print_top++;

            if (curFunc.paramkind[param_num - 1] == 3) { // 배열 기대 시 에러 추가
                push_err_list('e', last_func_line);
                break;
            }
        }
    }

    // 반복문 종료 후, nextToken 호출 횟수만큼 preToken 호출
    while (nextTokencnt > 0) {
        preToken();
        nextTokencnt--;
    }

    return;
}
```

错误类型F和G需要解决函数的返回值错误。我们通过创建全局变量 isneedreturn 和 isstmtreturn，并将其设为布尔类型，用来判断返回值的错误。

错误类型L和M也是较简单，基本逻辑是跟F和G一样

```
void parseErrorF(int linenum){
    if(!isneedreturn){
        push_err_list('f', linenum);
    }
}

void parseErrorG(){
    if(isneedreturn){
        if(!isstmtreturn){
            push_err_list('g');
        }
    }
}
```

```
void parseErrorL(string formatString, int print_param_cnt, int printlinenum) {
    int formatCount = 0;
    for (int i = 0; i < formatString.length(); i++) {
        if (formatString[i] == '%' && (formatString[i + 1] == 'd' || formatString[i + 1]
            formatCount++;
        }
    }

    if (formatCount != print_param_cnt) {
        push_err_list('l', printlinenum);
    }
}

void parseErrorM(){
    if(!isBorC){
        push_err_list('m');
    }
}
```

编码完成之后的修改

1.错误处理E

问题：出现了各种错误，如无限循环、设计失误、预期外的输出等。

修正：进行了修复逻辑和基本设计，并进行各种各样的testcase来测试代码

2.其他错误处理

问题：只是一些设计错误，跟错误处理E相比还是挺简单的。

修正：改了一些逻辑错误，并进行了优化，从而修复错误并提高效率

语义分析设计

设计总结

意义分析与代码生成

基于语法分析、语法规则与错误处理工作，我们实现了编译器的意义分析与代码生成功能。根据语法规则和语义规则，采用自底向上的语法指导翻译技术进行意义分析，并生成目标代码（PCODE）。

栈操作

栈顶指针初始值为 -1，每次将栈顶指针增加 1 (INT 1)。

然后，将相对地址存储到栈顶 (LDA 0 0)，并将分配的值存储到栈中 (LDC)。

将栈顶内容存储到之前标记的地址，然后栈减少两次 (STOS)，完成分配操作。

对于没有分配操作的语句，默认分配值为 0。

数组处理

数组只需要按顺序存储值，这与常量或变量的定义及基本原理相同。

计算

加法、减法、乘法、除法、取余等二元运算的计算通过以下三个步骤完成:

将第一个操作数压入栈 (LDC 1)。

将第二个操作数压入栈 (LDC 2)。

执行对应的运算 (ADD, SUB, MUL, DIV, MOD)。

例如，加法运算:

将第一个操作数 (LDC 1) 压入栈。

将第二个操作数 (LDC 2) 压入栈。

执行加法运算 (ADD)。

输入输出 (scanf, printf)

读操作：读取地址 (LDA x y 或 LDC x)，然后从该地址读取值。

写操作：读取地址 (LDA x y 或 LDC x)，然后向该地址写入值。

读写操作通过地址来处理数据，读取或写入相应的值。

条件语句与逻辑表达式

在条件语句中，首先计算条件块（cond block）的内容，将结果存储到栈顶。如果栈顶值为 1，进入 if 块；如果为 0，则进入 else 块。

在逻辑表达式中， 每次读取操作数时， 执行其求值， 支持短路求值 （short-circuit evaluation）。

例如，逻辑表达式 A && B:

先读取并求值 A。

如果 A 为 false，则跳过 B 的计算，逻辑表达式整体为 false。

如果 A 为 true，则读取并求值 B，然后根据 B 的值决定逻辑表达式的整体结果。

函数处理

在函数调用时，根据 Pcode 定义，在每次进入函数块时分配三个空间，分别用于存储返回值、返回值地址和返回值指令序列。

使用 call 指令跳转到函数内部进行计算。

函数结束时，将返回值的大小存储到栈顶。

对于数组调用，从栈中提取数组地址，在函数内使用该数组时确保不会出问题。

主要组成部分

1.Label class

```
class Label{
    private:
        int add = 0;

    public:
        Label() { }

        int getAdd() {
            return add;
        }

        void setAdd(int add) {
            this->add = add;
        }
};
vector<Label*> forbreak;
vector<Label*> forcontinue;
```

2.Code class

```
class Code{
    private:
        string name;
        int scope = 0;
        int addr = 0;
        string print = "";
        Label *label=nullptr;

        int type = 0;
    public:
        Code() {}

        Code(string name) : name(name), type(1){}
        Code(string name, int addr) : name(name), addr(addr), type(2){}
        Code(string name, string print) : name(name), print(print), type(3){}
        Code(string name, Label* label) : name(name), label(label), type(4){}
        Code(string name, int scope, int addr) : name(name), scope(scope), addr(addr), type(5){

        string getName(){
            return name;
        }

        int getScope(){
            return scope;
        }

        int getaddr(){
            return addr;
        }

        string getPrint(){
            return print;
        }

        Label* getLabel(){
            return label;
        }
};
vector<Code> codelist;
int PcodeAddress = 0;
```

3. Value class

```
class Value {
    private:
        int value;
        string Svalue;
        bool isInt;
    public:
        Value(int value) : value(value), isInt(true) {}
        Value(string Svalue) : Svalue(Svalue), isInt(false) {}

        int getValue() {
            if(isInt){
                return value;
            }

        }

        void setValue(int value){
            this->value = value;
            isInt = true;
        }

        string getSValue() {
            if(!isInt){
                return Svalue;
            }
        }

        void setSValue(string Svalue){
            this->Svalue = Svalue;
            isInt = false;
        }
};
```

Label是用于生成code的时候把label来表示code的位置。

Code是用于codelist里面加上该Pcode和需要的变量。

Value是为了计算常数变量设计的。

在生成代码里面主要的逻辑是设计Pcode的，但是代码量较大请参考代码里面的interpreter Class的 Pcode处理

编码完成之后的修改

1.函数的参数调用问题

问题：调用参数时出错

修正：在栈上推Pcode的时候Pcode的逻辑不对，改善了Pcode的逻辑bing进行优化

2.在函数里面处理字符串问题

问题：函数中调用字符串时，例如:有func(int a[])，Int arr1[10]的时候，直接func(arr1)这样，只有调用arr1[0]的地址，无调用剩下字符串的地址。

修正：所以直接用了字符串的原来的地址，不是在函数的栈里推新的地址，直接调用